

iSYST AKADEMIE

iTestStudio – Advanced Level (1) – Workshop

TESTHAUS AUS ÜBERZEUGUNG

© 2021 iSyst GmbH – alle Rechte vorbehalten

Preparation & Technology Check-In

to start the workshop

© 2021 iSyst GmbH – alle Rechte vorbehalten

Preparation

We will work with the Test Environment from last workshop (Basic Level)

→ Copy all tests from **TestPool** into your existing TestPool

→ Copy the folder **images** and **measurement** into **TestReports\workshop_testserie**

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Results

Basic and Extended Features

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Result

Recap

A test result consists of a description and a verdict

```
['<description text>', '<VERDICT>']
```

Defined standard verdicts are

- PASSED
- FAILED
- ERROR

Other verdicts: **INFO**, **TODO**, **also** custom verdicts or even a blank verdict

HINT



All other verdicts have no influence on test evaluation

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Results

Description Highlighting

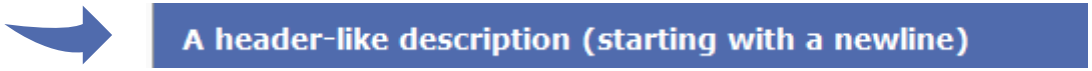
1. Bold description with `'\xa0'` (i.e. chr(160))

```
testresult.append(["\xa0A bold description (starting with a non-breakable "
                   "space)", "INFO"])
```



2. Header-like description with `'\n'`

```
testresult.append(["\nA header-like description (starting with a newline)", "PASSED"])
```



3. Bold description in another colour with `'´'`

```
testresult.append(["´Another bold description (starting with an acute "
                   "accent)", "FAILED"])
```



© 2021 iSyst GmbH – alle Rechte vorbehalten

Exercise #1: Test Result: highlighting

Open **05_testresult_features.py**

1. Add a header to the testresult with description „Testreport for different test result features“ (entry without verdict)

2. Add this file to your **workshop_testsequence**, which is inside the **workshop_testserie** (from Basic Level Workshop). Run the script.

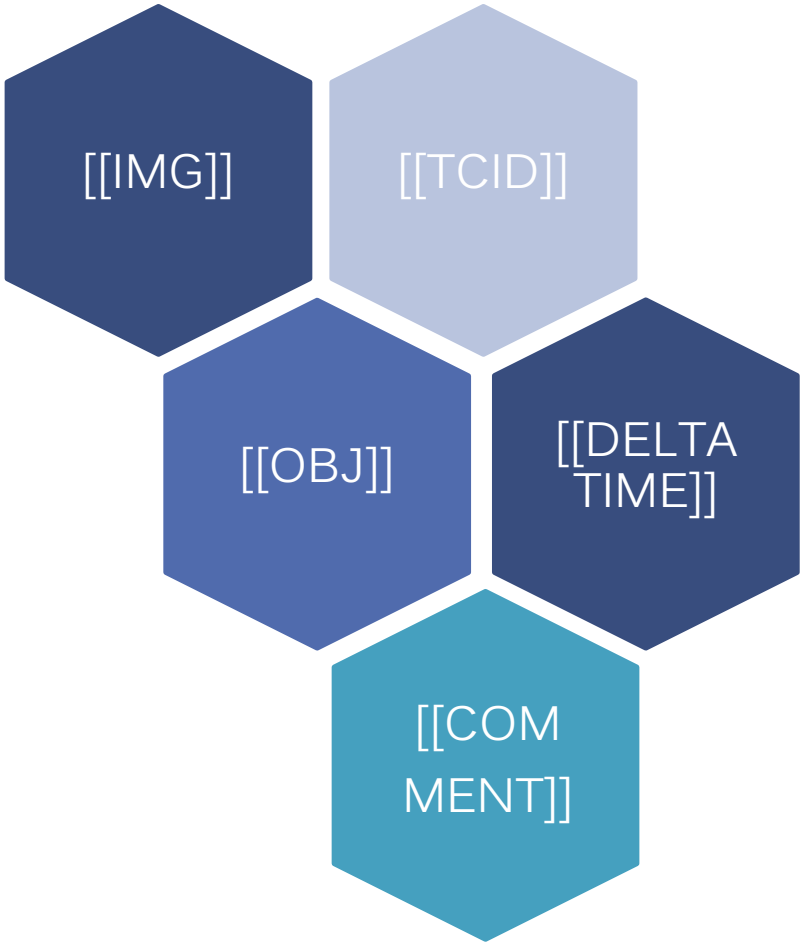
3. Complete the description with „features: highlights and extended format“ – write this in a new line inside the description

4. Add also testresults with the other highlight modes
   blue bold: „one simple image“
   green bold: „multiple images“

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Results

Extended Formats



© 2021 iSyst GmbH – alle Rechte vorbehalten

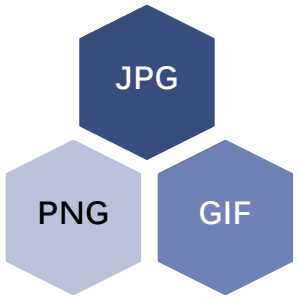
Test Results

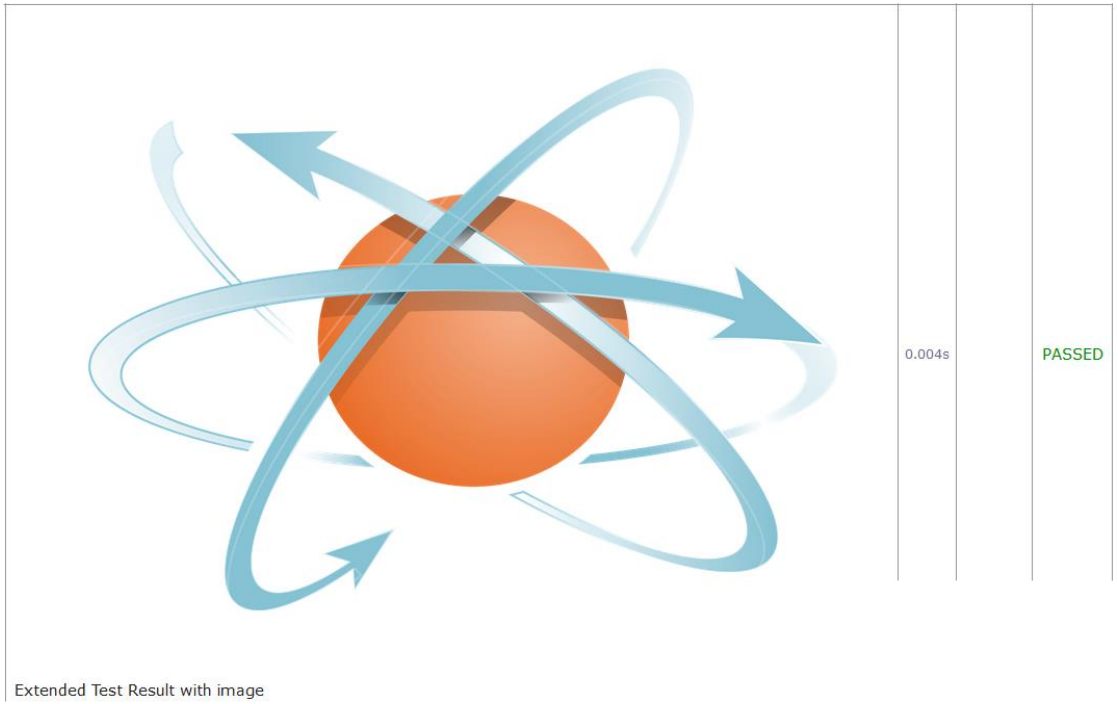
Extended Formats – Images

You can add images to your testresult

```
['<description text>', '[[IMG]] <path relative to xml-report>', '<VERDICT>']
```

```
testresult.append([
    "Extended Test Result with image",
    "[[IMG]] images/iSyst 360°.png",
    "PASSED"
])
```





© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Results

Extended Formats – Object

With an object you can add a references generic file path into the test result

```
['<description text>', '[[OBJ]] <path relative to xml-report>', '<VERDICT>']
```

```
testresult.append(["Add a object to the testresult",
                   "[[OBJ]] images/iSyst-Logo-mit-Slogan_rgb.tiff",
                   "PASSED"])
```



© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Results

Extended Formats – Testcase ID

You can also add a testcase id to a single testresult.

```
['<description text>', '[[TCID]] <tc id>', '<VERDICT>']
```

```
descr, verdict = basic_tests.checkTolerance(
    hil.motor_temp, excepted_value, 0, 0.1,
    descr="Check initial motor temperature"
    )

testresult.append([descr, "[[TCID]] tr123", verdict])
```



Check initial motor temperature Expected value: 20.0 °C Current value: 20.0 °C (motor_temp) Absolute Tolerance: +/- 0.1 °C	0.398s	tr123	PASSED

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Results

Extended Formats – Comment

With an comment you can add a note to the test result

```
['<description text>', '[[COMMENT]] <comment text>', '<VERDICT>']
```

```
descr, verdict = basic_tests.checkTolerance(
    hil.motor_temp, excepted_value, 0, 0.1,
    descr="Check initial motor temperature"
    )

testresult.append([descr, "[[COMMENT]] ticket no: 12345", verdict])
```



Check initial motor temperature Expected value: 20.0 °C Current value: 10.0 °C (motor_temp) Absolute Tolerance: +/- 0.1 °C	0.227s		**FAILED** ticket no: 12345

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Results

Extended Formats – Deltatime

You can also add a new deltatime (as seconds since test start) to the testresult

```
['<description text>', '[[DELTATIME]] <seconds>', '<VERDICT>']
```

```
descr, verdict = basic_tests.checkTolerance(
        hil.motor_temp, 20.0, 0, 3,
        descr="Check motor temperature"
        )

testresult.append([descr, "[[DELTATIME]] 12.345", verdict])
```



Check motor temperature Expected value: 20.0 °C Current value: 10 °C (motor_temp) Absolute Tolerance: +/- 3 °C	12.345s		**FAILED**

HINT



Normally, timestamps are handled by the **ResultList** utility class, the manually setting of these is not necessary

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Results

Extended Formats

```
['<normal description text>', '[[<KEYWORD>]] <data>', '<VERDICT>']
```

Further information:

- You can add a keyword more then one time
- You can add different keywords

```
['<normal description text>', '[[<KEYWORD>]] <data>',
 '[[<KEYWORD>]] <data>', …, '[[<KEYWORD>]] <data>', '<VERDICT>']
```

HINT



If more than one delta time has been provided (for whatever reason), the last entry will be used.

© 2021 iSyst GmbH – alle Rechte vorbehalten

Exercise #2: Test Result: extended

Open **05_testresult_features.py**

1. Add the image **dice.cc0.jpg** into a separate test result below the header **one simple image**
2. Add the images **plot.png** and **some_plotted_data.png** into a separate testresult below the header **multiple images**
3. Add a test step with header **„Check the motor temperature“**. The **motor_temp** shall be 20°C ± 3°C (ttk-function!). Add the test case ID **mo885**.
4. Add a comment to the testresult of the motor_temp check. If the verdict is „FAILED“ following text shall added: **known issue, ticket no: 3698**
5. Add a loop around the motor_temp teststep, in first loop the **motor_temp** shall be set to 16°C, in the second loop 17°C

→ Check the html report of the test script, after a run in iTestStudio

© 2021 iSyst GmbH – alle Rechte vorbehalten

Exercise #3: Test Result: deltatime

Open **06_testresult_deltatime.py**

1. Add a header with **Start testrun for deltatime** add verdict „INFO“
2. Add the **eel_distribution.mat** from folder **measurement** as a object, with verdict „INFO“
3. Add a teststep with description **simple teststep with no further information**, the verdict shall be „PASSED“. Add a deltatime of 12.445.
4. Add a second deltatime of 11.123 to the teststep
5. Add the testscript to the **workshop_testsequence.seq** and run this in iTestStudio
6. Remove line 26 (testenv.testresult = []) and rerun

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Results

ResultList Features

© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Test Result Echo 1/2

By default **every new test result** will be echoed on the log console.
You can disable the echo with following function:

```
testresult.enableEcho(False)
```

All results AFTER this command will not be shown in the console (but will still appear in the report)

During **testenv.breakdown()** an overview list of all results will be created.
If you want to disable this output, you have set additional:

```
testenv.show_testresults = False
```

HINT



Settings have no influence on the test report!

© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Test Result Echo 2/2

```
setup is done!


################################################################################
# Header like Description with '\n' at the beginning
################################################################################
  INFO   - Simple Test Results with INFO verdict
           Delta time: 0.01s


################################################################################
# Simple Test Results without verdict
################################################################################
 PASSED - Simple Test Results with PASSED verdict
          Delta time: 0.01s
 FAILED - Another simple Test Results with FAILED verdict
          Delta time: 0.01s
Resetting... ...done.

[[breakdown]]

--Elapsed Time:---------------
  0.0 seconds

--Results:--------------------
# \nHeader like Description with '\n' at the beginning
   INFO   - Simple Test Results with INFO verdict
# Simple Test Results without verdict
  PASSED  - Simple Test Results with PASSED verdict
 >FAILED< - Another simple Test Results with FAILED verdict

--Summary:--------------------
  INFO   : 1
  FAILED : 1
  PASSED : 1
------------------------------
```

Testrun with echo enabled (by default)

```
############################################################
setup is done!

Resetting... ...done.

[[breakdown]]

--Results:--------------------
# \nHeader like Description with '\n' at the beginning
   INFO   - Simple Test Results with INFO verdict
# Simple Test Results without verdict
  PASSED  - Simple Test Results with PASSED verdict
 >FAILED< - Another simple Test Results with FAILED verdict

--Summary:--------------------
  INFO   : 1
  FAILED : 1
  PASSED : 1
------------------------------
```

testrun with echo disabled

```
setup is done!

Resetting... ...done.

  0.0 seconds

--Summary:--------------------
  INFO   : 1
  FAILED : 1
  PASSED : 1
------------------------------
```

also result overview echo disabled

© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Delta Time 1/2

The ResultList will automatically add a **delta time** entry to **every test result** with an verdict.
The delta time will be shown in the test report and also for every test result on the log console.

This can be disabled by using following function:

```
testresult.enableDeltaTime(False)
```

All results AFTER this command will be logged without a deltatime.

HINT



This influences the test report (delta times will be missing)!

© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Delta Time 2/3

```
################################################################################
# Header like Description with '\n' at the beginning
################################################################################
  INFO  - Simple Test Results with INFO verdict
        Delta time: 0.01s


################################################################################
# Simple Test Results without verdict
################################################################################
 PASSED - Simple Test Results with PASSED verdict
        Delta time: 0.52s
 FAILED - Another simple Test Results with FAILED verdict
        Delta time: 0.73s
Resetting... ...done.

[[breakdown]]

--Elapsed Time:----------------
  0.7 seconds
```

Δ time enabled

```
################################################################################
# Header like Description with '\n' at the beginning
################################################################################
  INFO  - Simple Test Results with INFO verdict


################################################################################
# Simple Test Results without verdict
################################################################################
 PASSED - Simple Test Results with PASSED verdict
 FAILED - Another simple Test Results with FAILED verdict
Resetting... ...done.

[[breakdown]]

--Elapsed Time:----------------
  0.7 seconds
```

Δ time disabled

© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Delta Time 3/3

	Δt	TC-ID	Verdict
Header like Description with '\n' at the beginning	0.000s		
Simple Test Results with INFO verdict	0.009s		INFO
Simple Test Results without verdict			
Simple Test Results with PASSED verdict	0.514s		PASSED
Another simple Test Results with FAILED verdict	0.729s		FAILED

Δ time enabled

	Δt	TC-ID	Verdict
Header like Description with '\n' at the beginning	0.000s		
Simple Test Results with INFO verdict			INFO
Simple Test Results without verdict			
Simple Test Results with PASSED verdict			PASSED
Another simple Test Results with FAILED verdict			FAILED

Δ time disabled

© 2021 iSyst GmbH – alle Rechte vorbehalten

Exercise #4: ResultList Echo

Open **07_testresult_resultlist.py**

Add following teststeps to the script:

- Header with description: **Start testrun for motor temperature**
- set **motor_temp** to 21 (°C) and add a waiting time of 0.5 seconds
- Add a TTk test function (TestToolkit):
  the expected **motor_temp** shall be between 15°C and 25°C, add a suitable description – add this fucntion‘s output to testresults
- Add an ‚INFO‘ testresult in bold with description: **the clamp 30 has to be switched on** and add a further waiting time of 0.2 seconds
- Add a further TTk test function and check that the status of **clamp_30** shall be 1, add a suitable description and add this also to testresults

→ Run the script in your test environment (e. g. Eclipse) and check the output

- Add **testresult.enableEcho(False)** before your test starts and check output again
- Add **testenv.show_testresults = False** and check the output again

© 2021 iSyst GmbH – alle Rechte vorbehalten

Exercise #5: ResultList Deltatime

Open **07_testresult_resultlist.py**

- Remove the „enable Echo“ steps from script from exercise before
- Add the test to your **workshop_testsequence.seq** in iTestStudio and run the test in iTestStudio. Create a report and check the output
- Add **testresult.enableDeltaTime(False)** to the script and run it again in Eclipse and in iTestStudio

© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Test Case IDs

You can add a automatically set test case ID to all following test results:

```
testresult.setTestcaseId('ID')
```

HINT



- Only one test case ID can be activated with this function (before iTS 5.1.0, see release notes)
- Setting a new test case ID will replace the previous ID for the following results

You can stop automatically assigning IDs by using this function:

```
testresult.clearTestcaseId()
```


© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Multiple test results

With the automatically set test case ID, only a single ID can be set.

If you want to set a further ID (or want to set an additional ID without changing the auto-TC-ID setting), you can to use the extended format [[TCID]].

```
testresult.setTestcaseId('<ID>')
```

```
# a single test case ID
testresult.append([description, "[[TCID]] <Further ID>", verdict])
```



	Δt	TC-ID	Verdict
description	0.010s	<ID> <Further ID>	verdict

© 2021 iSyst GmbH – alle Rechte vorbehalten

Exercise #6: TestcaseID

Open **07_testresult_resultlist.py** and remove the EnableDeltaTime adaption from exercise #5

- Set automatically added test case ID **TC007** before the first verdict will be added to the testresults
- The clamp 30 teststep shall have a further test case ID: **TC001**
- Add a new teststep at the end:
  Check that the **clamp_15** is also in status 1 (active). Add this step to the testresults

→ Run the test in iTestStudio and check the report

- After the range test of **motor_temp** add the step to clear the test case ID (clearTestcaseId())

→ Run the test in iTestStudio and check the report

- Instead of the clearTestcaseId-step add the setTestcaseID and write a new id: **TC123**

→ Run the test in iTestStudio and check the report

© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Automatic Chapter Numbering 1/3

You can add a chapter number manually or use the chapter numbers for ResultLust, which can automatically increment/decrement

```
['[<chapter cmd>] <actual header text>', ''] # verdict must be empty!
```

Chapter Commands:

[#<nr>]	Set current chapter number
[.]	Continue with next chapter on the current level
[+]	Go one sub chapter down, add a new sub chapter
[-]	Go one chapter level up, add next parent chapter
[+0]	Go one sub chapter down, add new sub chapter but start a 0. This will not add an header entry in the test results. Use [.] to start the first sub chapter.
[-0]	Go one chapter level up, back to the current parent chapter This will not add an header entry in the test results (see [+0]).

© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Automatic Chapter Numbering 2/3

Add into your script:

```
testresult.append(["[.] Start with chaptering", ""])
testresult.append(["[+] Go one sub chapter down, add a new sub chapter", ""])
testresult.append(["[-] Go one chapter level up, add next parent chapter", ""])
```

Result in the test environment console:

```
--Results:--------------------
# [1] Start with chaptering
# [1.1] Go one sub chapter down, add a new sub chapter
# [2] Go one chapter level up, add next parent chapter
```

Output in test report:

No.		Δt	TC-ID
1	[1] Start with chaptering	0.000s	
2	[1.1] Go one sub chapter down, add a new sub chapter		
3	[2] Go one chapter level up, add next parent chapter		

© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Automatic Chapter Numbering 3/3

You can also change following default settings of the chaptering:

- Start chapter
- Separator
- Number format
- Wrapper

This will be handled by an instance of ChapterNumbers

```
from result_list import ChapterNumbers  # @UnresolvedImport (in .pyz)
# [...]
testresult.current_chapter = ChapterNumbers(
    start_chapter="3", separator="-",
    number_format="%02d", wrapper="(Episode %s)"
)
```


© 2021 iSyst GmbH – alle Rechte vorbehalten

Exercise #7: Chapter Numbering

Open **08_testresult_chapter.py**

All chapter are manually added!
→ Change now the script, so that the chaptering of ResultList will be used!
The output shall be equal to the manual chapters.

Now change the default settings by use **ChapterNumbers**:
start_chapter = „1“, **separator** = „.“,
wrapper = „\n[Chapter %s]“

© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Combined Sub-Results

You can convert multiple sub-results into one single result entry:

1. Create a separat ResultList()

```
subresult = ResultList()
```

2. (optional) Disable the Echo to not echo the results twice

```
subresult.enableEcho(False) # no need to echo results twice
```

3. Add the sub-results into the new ResultList()

```
subresult.append(["A passed sub-result",        "PASSED"])
subresult.append(["A failed sub-result",        "FAILED"])
subresult.append(["An info-level sub-result",      "FOO"])
```

4. Add the combined sub-results into the testresults

```
testresult.append(subresult.getCombinedResult())
```


© 2021 iSyst GmbH – alle Rechte vorbehalten

ResultList

Subresults

Output in the HTML report:

54	31	**Combined Sub-Results**			
55	32	A passed sub-result --- > A failed sub-result --- An info-level sub-result	0.328s	SUB-01 RESV-01	**FAILED**

The divider between the sub-results and the prefix for failed results is configurable:

```
subresult.getCombinedResult(divider="\n", failed_prefix="?! ")
```


© 2021 iSyst GmbH – alle Rechte vorbehalten

Exercise #8: subresults

Open **07_testresult_resultlist.py**

- Convert the **clamp_30** and **clamp_15** teststep into a combined sub-result

→ Run the test in iTestStudio and show the testreport

- Change the **divider** of this subresults into **„\n.....\n“**
- Change the **failed_prefix** into **„-->“**

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Environment

Default Methods

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Environment

TestEnvironmentBase-Methods

Method	description
testenv.setMetaEntry("name", "value")	set an (additional) meta entry to be stored along with test results

```
testenv.setMetaEntry("new_meta_entry", "value of new meta entry")
```



```
<meta>
    <title>07 testresult resultlist</title>
    <description>test for different features</description>
    <author>An3Neumann</author>
    <date>09.03.2021</date>
    <version>1.0</version>
    <last_revision>initial</last_revision>
    <starttime>2021-04-16T08:54:38</starttime>
    <endtime>2021-04-16T08:54:39</endtime>
    <custom_meta_name>Custom Meta Value</custom_meta_name>
    <hil_computername>ISYCNB104</hil_computername>
    <hil_project_path>D:\HIL-Projekte\HIL-Workshop</hil_project_path>
    <hil_username>An3Neumann</hil_username>
    <new_meta_entry>value of new meta entry</new_meta_entry>
</meta>
```


© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Environment

TestEnvironmentBase-Methods

Method	description
testenv.setMetaEntry("name", "value")	set an (additional) meta entry to be stored along with test results
testenv.getResults()	get a reference to the test results list

```
>>> testenv.getResults()

[['\nStart testrun for motor temperature', ''], [u'Check that motor temperature
is between 15\xb0C and 25\xb0C\nValue:     21\nMin value: 15\nMax value: 25', '[
[DELTATIME]] 0.510', u'[[TCID]] TC007', 'PASSED'], ['\xa0the clamp 30 has to be
switched on', '[[DELTATIME]] 0.511', 'INFO'], [u'-->Check that clamp 30 status i
s active\n-->Current status:  0.0 [off] (CL30)\n-->Expected status: 1 [on]\n....
.........\n-->Check that clamp 15 status is active\n-->Current status:  0.0 [of
f] (CL15)\n-->Expected status: 1 [on]', u'[[TCID]] TC001', '[[DELTATIME]] 0.715'
, u'[[TCID]] TC123', 'FAILED']]
```


© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Environment

TestEnvironmentBase-Methods

Method	description
testenv.setMetaEntry("name", "value")	set an (additional) meta entry to be stored along with test results
testenv.getResults()	get a reference to the test results list
testenv.printResults()	print all currently available test results

```
>>> testenv.printResults()

--Results:--------------------
# \nStart testrun for motor temperature
  PASSED  - Check that motor temperature is between 15°C and 25°C\nValue:     21
\nMin value: 15\nMax value: 25
   INFO   -  the clamp 30 has to be switched on
 >FAILED< -  Check that clamp 30 status is active\nCurrent status:  0.0 [off] (CL
30)\nExpected status: 1 [on]
 >FAILED< -  Check that clamp 15 status is active\nCurrent status:  0.0 [off] (CL
15)\nExpected status: 1 [on]
```


© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Environment

TestEnvironmentBase-Methods

Method	description
testenv.setMetaEntry("name", "value")	set an (additional) meta entry to be stored along with test results
testenv.getResults()	get a reference to the test results list
testenv.printResults()	print all currently available test results
testenv.getResultCount()	get a count of currently present result verdicts (as a dictionary with <verdict>: <count> entries)

```
>>> testenv.getResultCount()

{u'INFO': 1, 'FAILED': 2, 'PASSED': 1}
```


© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Environment

TestEnvironmentBase-Methods

Method	description
testenv.setMetaEntry("name", "value")	set an (additional) meta entry to be stored along with test results
testenv.getResults()	get a reference to the test results list
testenv.printResults()	print all currently available test results
testenv.getResultCount()	get a count of currently present result verdicts (as a dictionary with <verdict>: <count> entries)
testenv.printResultsSummary()	print a summary of current test results

```
>>> testenv.printResultsSummary()

--Summary:---------------------
  INFO   : 1
  FAILED : 2
  PASSED : 1
-------------------------------
```


© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Environment

TestEnvironmentBase-Methods

Method	description
testenv.setMetaEntry("name", "value")	set an (additional) meta entry to be stored along with test results
testenv.getResults()	get a reference to the test results list
testenv.printResults()	print all currently available test results
testenv.getResultCount()	get a count of currently present result verdicts (as a dictionary with <verdict>: <count> entries)
testenv.printResultsSummary()	print a summary of current test results
testenv.getResultsFilePath()	get a path to the configured result file
testenv.storeResults()	store current test results to the result file
testenv.getElapsedTime()	get seconds since start of test script (or, more accurately, since TestEnv has been instantiated)

```
>>> testenv.getElapsedTime()

221.89200019836426
```


© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Environment

TestEnvironmentBase-Methods

Method	description
testenv.setMetaEntry("name", "value")	set an (additional) meta entry to be stored along with test results
testenv.getResults()	get a reference to the test results list
testenv.printResults()	print all currently available test results
testenv.getResultCount()	get a count of currently present result verdicts (as a dictionary with <verdict>: <count> entries)
testenv.printResultsSummary()	print a summary of current test results
testenv.getResultsFilePath()	get a path to the configured result file
testenv.storeResults()	store current test results to the result file
testenv.getElapsedTime()	get seconds since start of test script (or, more accurately, since TestEnv has been instantiated)
testenv.printElapsedTime()	print elapsed time since test start

```
>>> testenv.printElapsedTime()
--Elapsed Time:----------------
  3 minutes, 53.0 seconds
```


© 2021 iSyst GmbH – alle Rechte vorbehalten

Exercise #9: Default Methods

Open **07_testresult_resultlist.py**

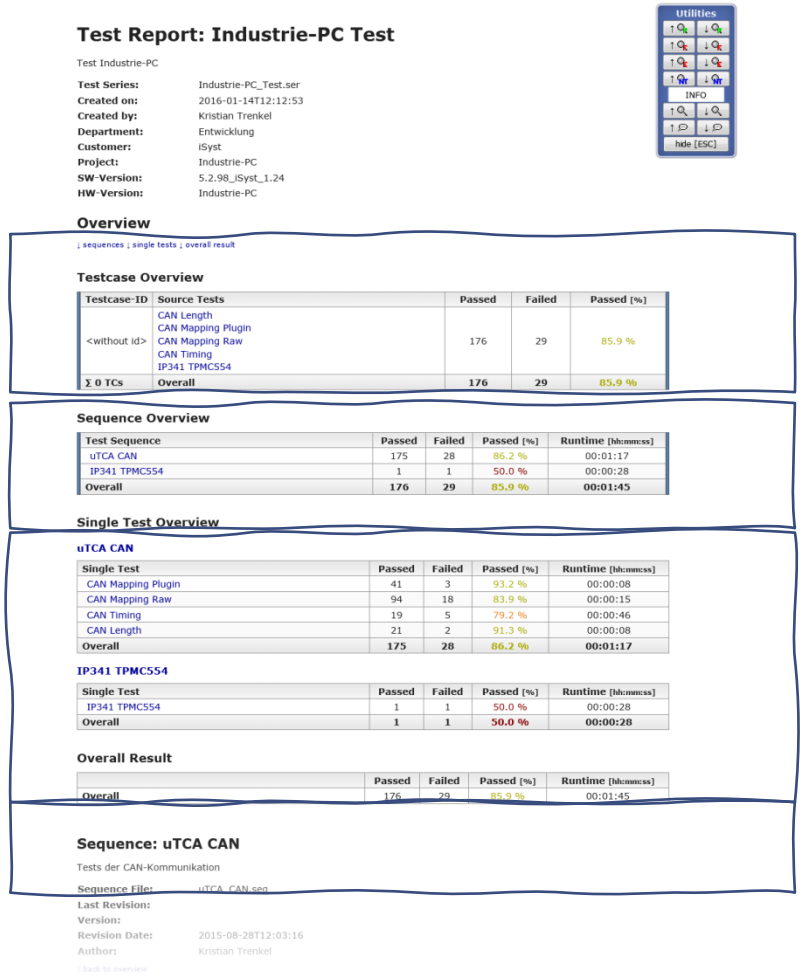
- At the end of the test read out the elapsed time and store this as variable **elapsed_time**
- Add a new meta entry with name **test_duration** and value **elapsed_time** → run the script (in Eclipse) and check the xml (...\TestReports\SingleTestResults\SingleTestManual - 07_testresult_resultlist.xml)
- Read out the number of results (getResultCount())
  - store the number of INFO-results as **info_count**
  - the number of PASSED-results as **passed_count**
  - the number of FAILED-results as **failed_count**
  - print all the stored results: „<kind of result>: <count of result>“
- print the ResultsSummary → run the script (in Eclipse), the number of results should be equal to the printed results from step before

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Reporting

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Reporting



Test Report: Industrie-PC Test

Test Industrie-PC

Test Series:	Industrie-PC_Test.ser
Created on:	2016-01-14T12:12:53
Created by:	Kristian Trenkel
Department:	Entwicklung
Customer:	iSyst
Project:	Industrie-PC
SW-Version:	5.2.98_iSyst_1.24
HW-Version:	Industrie-PC

Utilities
INFO
hide [ESC]

Overview

↓ sequences ↓ single tests ↓ overall result

Testcase Overview

Testcase-ID	Source Tests	Passed	Failed	Passed [%]
<without id>	CAN Length CAN Mapping Plugin CAN Mapping Raw CAN Timing IP341 TPMC554	176	29	85.9 %
Σ 0 TCs	Overall	176	29	85.9 %

Sequence Overview

Test Sequence	Passed	Failed	Passed [%]	Runtime [hh:mm:ss]
uTCA CAN	175	28	86.2 %	00:01:17
IP341 TPMC554	1	1	50.0 %	00:00:28
Overall	176	29	85.9 %	00:01:45

Single Test Overview

uTCA CAN

Single Test	Passed	Failed	Passed [%]	Runtime [hh:mm:ss]
CAN Mapping Plugin	41	3	93.2 %	00:00:08
CAN Mapping Raw	94	18	83.9 %	00:00:15
CAN Timing	19	5	79.2 %	00:00:46
CAN Length	21	2	91.3 %	00:00:08
Overall	175	28	86.2 %	00:01:17

IP341 TPMC554

Single Test	Passed	Failed	Passed [%]	Runtime [hh:mm:ss]
IP341 TPMC554	1	1	50.0 %	00:00:28
Overall	1	1	50.0 %	00:00:28

Overall Result

	Passed	Failed	Passed [%]	Runtime [hh:mm:ss]
Overall	176	29	85.9 %	00:01:45

Sequence: uTCA CAN

Tests der CAN-Kommunikation

Sequence File:	uTCA_CAN.seq
Last Revision:	
Version:	
Revision Date:	2015-08-28T12:03:16
Author:	Kristian Trenkel

↑ back to overview

Navigation elements

Testcase Overview

Sequence Overview

Single Test Overview

Overall Overview

© 2021 iSyst GmbH – alle Rechte vorbehalten

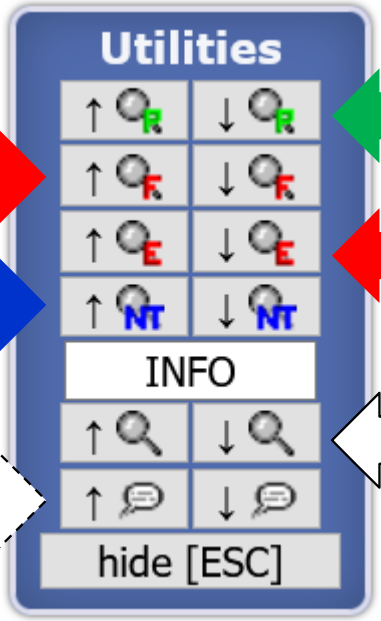
Test Reporting

Testreport Navigation in HTML

go forward/backward to next FAILED Result

go forward/backward to next „no“ Result

go forward/backward to next comment in Test Results



go forward/backward to next PASSED Result

go forward/backward to next ERROR Result

go forward/backward to next defined verdict (default = INFO)

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Reporting

Configuration of report – Overview

```
#################################################################
# Overview
#
# Show Sequence Overview (results grouped by sequence, linked
# to matching entries in Single Test Overview)
show_sequence_overview = yes

# Show Testcase Overview
# (results grouped by testcase-id)
show_testcase_overview = yes

# currently only for testing purposes:
# Show Sequence Testcase Overview
# (testcase-id-results grouped by sequence)
show_seq_testcase_overview = no

# Show script runtime info in overview:
# Note that scripts without (valid) start/end time information
# will show a default of n/a as runtime
show_singletest_runtimes = yes

# show additional verdicts in sequence/test overview (i.e. additionally to
# PASSED / FAILED). Add multiple verdicts separated by comma:
show_additional_verdicts = TODO, INFO, FOO

# show additional verdicts in testcase overview (i.e. additionally to
# PASSED / FAILED). Add multiple verdicts separated by comma:
show_additional_verdicts_tco = ERROR
```

The overview contents can be configurated in the **html_report.ini** (..\iTestStudio\settings\)

© 2021 iSyst GmbH – alle Rechte vorbehalten

Exercise #10: html-report

- Run the workshop_testserie.ser and have a look at the html report. Check which parts are shown.

- Open the html-report.ini and change the following settings (after every change re-create the report):
  - set show_testcase_overview to no
  - set show_sequence_overview to no
  - set show_seq_testcase_overview to yes
  - set show_singletest_runtimes to no
  - comment in show_additional_verdicts
  - Add INFO to show_additional_verdicts_tco
  - set show_testcase_overview back to yes

© 2021 iSyst GmbH – alle Rechte vorbehalten

Test Reporting

Configuration of report – Meta entries

```
# Meta entries in single tests to be shown (in order of appearance)
# Defaults: LAST_REVISION, VERSION, DATE, AUTHOR
# Any field in meta info can be shown here. To get a "nicer"
# localization of the label, it has to be added to the L10N-data,
# the meta entry key will be used as fallback.
#
# singletest_meta_entries = Last_Revision, Version, Date, Author
```

Test Source File:	07_testresult_resultlist.py
Tested on:	2021-04-16T08:52:26
Last Revision:	initial
Version:	1.0
Revision Date:	09.03.2021
Author:	An3Neumann
Testcase-IDs:	TC001, TC007, TC123

© 2021 iSyst GmbH – alle Rechte vorbehalten

Exercise #11: html-report meta entries

Open html_report.ini

In the python script 07_testresult_resultlist.py you have added:

```
testenv.setMetaEntry("test_duration", elapsed_time)
```

- Now add this output to the meta entries in your html test report
- Run the script again and check the meta entries of the single test

© 2021 iSyst GmbH – alle Rechte vorbehalten



iSYST AKADEMIE

Thank you for your attention!

TESTHAUS AUS ÜBERZEUGUNG

© 2021 iSyst GmbH – alle Rechte vorbehalten